

## Metodología de Sistemas II

### Trabajo Práctico Adapter

Refactoring en el mundo real - Migración de proveedor con APIs incompatibles

**Alumno:**

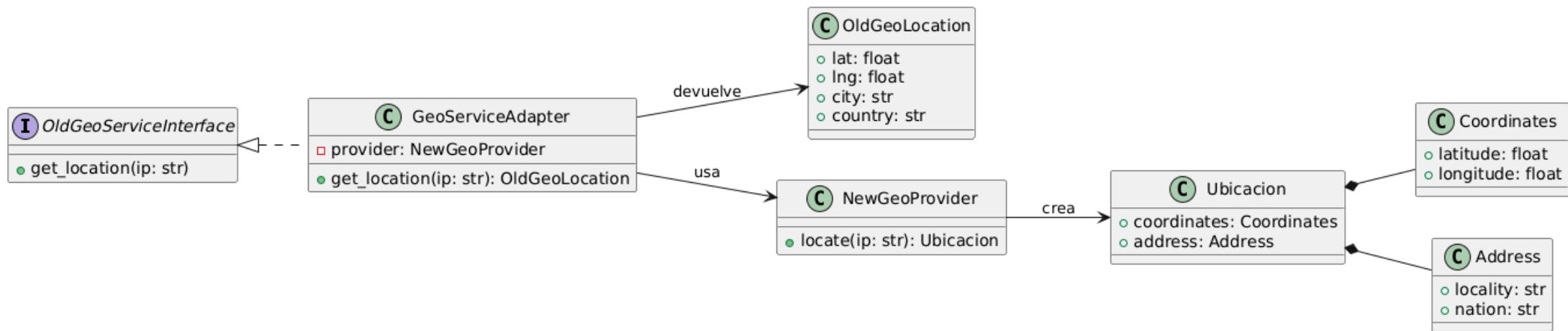
Matias Carro - [matiasmanuelcarro@gmail.com](mailto:matiasmanuelcarro@gmail.com)

DNI: 37362003

Legajo: 18743

|                                                                           |          |
|---------------------------------------------------------------------------|----------|
| <b>Diagrama uml.....</b>                                                  | <b>2</b> |
| <b>Antes / Después del Adapter.....</b>                                   | <b>3</b> |
| Antes (sin Adapter).....                                                  | 3        |
| Después (con Adapter).....                                                | 3        |
| <b>Justificación del uso de Adapter.....</b>                              | <b>4</b> |
| <b>¿Qué tendría que cambiar si llega un tercer proveedor mañana?.....</b> | <b>4</b> |
| <b>Código del sistema.....</b>                                            | <b>5</b> |
| Antes:.....                                                               | 5        |
| Después:.....                                                             | 6        |

## Diagrama uml



# Antes y Después del Adapter

## Antes (sin Adapter)

Antes de aplicar el patrón Adapter, el sistema viejo llamaba a un servicio que devolvía directamente los datos en el formato simple que esperaba:

```
geo = OldGeoService()
data = geo.get_location("200.45.123.10")

print("[LEGACY] Ciudad:", data.city, "| Latitud:", data.lat)
```

El problema con el cambio de API es que el proveedor nuevo (NewGeoProvider) devuelve una estructura completamente distinta:

```
Ubicacion(
    coordinates = Coordinates(latitude, longitude),
    address = Address(locality, nation)
)
```

El código viejo no entiende esta estructura. Si intentara usar el proveedor nuevo directamente, habría que modificar todos los archivos antiguos lo que llevaría un trabajo muy extenso.

## Después (con Adapter)

Con el Adapter, los archivos viejo siguen utilizando el mismo código:

```
geo = OldGeoService()
data = geo.get_location("200.45.123.10")

print("[LEGACY] Ciudad:", data.city, "| Latitud:", data.lat)
```

La diferencia es que ahora:

- `OldGeoService()` devuelve un `GeoServiceAdapter`.
- El Adapter llama internamente al proveedor nuevo.
- Traduce la estructura nueva al formato viejo.
- Devuelve un `OldGeoLocation` compatible con el sistema legacy.

El código viejo no se modifica, no da error y no necesita conocer la API nueva.

El adapter es el que realiza la traducción entre los formatos viejos y nuevos:  
Pudiendo así adaptar el formato nuevo de objetos al sistema antiguo.

```
return OldGeoLocation(  
    lat=ubicacion.coordinates.latitude,  
    lng=ubicacion.coordinates.longitude,  
    city=ubicacion.address.locality,  
    country=ubicacion.address.nation  
)
```

## Justificación del uso de Adapter

Se utiliza el patrón Adapter porque permite integrar un proveedor nuevo sin modificar el código legacy, manteniendo la firma original `get_location(ip)` y el formato de salida esperado. Una refactorización masiva rompería compatibilidad y obligaría a actualizar todos los módulos antiguos. Una fachada no resuelve el problema, porque solo simplifica una API, pero no traduce estructuras incompatibles. El Adapter actúa como una capa de traducción entre dos modelos distintos, aislando el cambio y garantizando que el sistema viejo continúe funcionando sin alteraciones.

## ¿Qué tendría que cambiar si llega un tercer proveedor mañana?

Si mañana llega un tercer proveedor, lo único que debe modificarse es el Adapter. El sistema legacy sigue llamando a `get_location(ip)` y continúa recibiendo un `OldGeoLocation`, por lo que no requiere ningún cambio. El nuevo proveedor puede tener otra estructura, otros nombres de métodos o incluso otro modelo de datos, pero el Adapter es la única pieza encargada de traducir esa nueva API al formato viejo. Mientras la firma `get_location(ip)` se mantenga idéntica y el Adapter realice la conversión correcta, el resto del sistema permanece intacto. Esto confirma que el patrón Adapter está bien aplicado.

# Código del sistema

Antes:

```
# ANTES
# LIBRERÍA VIEJA
class OldGeoService:
    def get_location(self, ip: str) -> dict:
        # Simulacion de algo que devolveria el OldGeoService
        return {
            "lat": -34.6037,
            "lng": -58.3816,
            "city": "Buenos Aires",
            "country": "Argentina"
        }

def test_antes():
    geo = OldGeoService()
    data = geo.get_location("200.45.123.10")
    print("[ANTES] Ciudad:", data["city"], "| Latitud:", data["lat"])

if __name__ == "__main__":
    test_antes()
```

Después:

```
# INTERFAZ DEL SERVICIO VIEJO
class OldGeoServiceInterface:
    def get_location(self, ip: str):
        # tiene que devolver lat, lng, city, country
        raise NotImplementedError("Debe implementarse en la subclase")

# NUEVA API (Proveedor Moderno)
class Coordinates:
    def __init__(self, lat: float, lng: float):
        self.latitude = lat
        self.longitude = lng

class Address:
    def __init__(self, locality: str, nation: str):
        self.locality = locality
        self.nation = nation

class Ubicacion:
    def __init__(self, lat: float, lng: float, locality: str, nation: str):
        self.coordinates = Coordinates(lat, lng)
        self.address = Address(locality, nation)

class NewGeoProvider:
    def locate(self, ip: str) -> Ubicacion:
        # Simulación de proveedor moderno
        return Ubicacion(-34.6037, -58.3816, "Buenos Aires", "Argentina")

# ADAPTER
class GeoServiceAdapter(OldGeoServiceInterface):
    def __init__(self, provider: NewGeoProvider):
        self.provider = provider
```

```

def get_location(self, ip: str) -> dict:
    ubicacion = self.provider.locate(ip)

    # Traducción al formato viejo
    return {
        "lat": ubicacion.coordinates.latitude,
        "lng": ubicacion.coordinates.longitude,
        "city": ubicacion.address.locality,
        "country": ubicacion.address.nation
    }

# FACTORY PARA EL CÓDIGO ANTIGUO (Legacy)

def OldGeoService():
    #El código viejo sigue llamando al OldGeoService() sin enterarse del cambio.
    return GeoServiceAdapter(NewGeoProvider())

# CLIENTE ANTES Y DESPUÉS

#Se puede ver como los archivos viejos pueden seguir funcionando sin enterarse de los
cambios
#Tambien los archivos nuevos pueden directamente con el nuevo servicio o el adapter

#Test con el codigo viejo (funciona usando el adapter)
# Los archivos viejos no necesitan ningun tipo de modificacion
def test_antes():
    geo = OldGeoService() #Siguen llamando a OldGeoService()
    data = geo.get_location("200.45.123.10")
    print("[ANTES] Ciudad:", data["city"], "| Latitud:", data["lat"])

#Test con el adapter (se puede utilizar el adapter directamente - solo como demostracion)
def test_adapter():
    geo = GeoServiceAdapter(NewGeoProvider())
    data = geo.get_location("200.45.123.10")
    print("[ADAPTER - DESPUES] Ciudad:", data["city"], "| Latitud:", data["lat"])

```

Test llamando al nuevo provider directamente (un archivo nuevo puede llamar a la api nueva, sin necesidad de usar adapter)

```
"""
def test_nuevo_servicio():
    geo = NewGeoProvider()
    ubicacion = geo.locate("200.45.123.10")
    print("[NEWGEOPROVIDER] Ciudad", ubicacion.address.locality, "| Latitud: ",
ubicacion.coordinates.latitude)
```

```
if __name__ == "__main__":
    test_antes()
    test_adapter()
    test_nuevo_servicio()
```